

DOCUMENTO TÉCNICO

Simulador de Escalonamento de Processos

Classes, estruturas de dados, algoritmos e decisões de implementação.

Marcos Vinicius Pereira Araujo

Tiago Melo Jucá

Vitor Nogueira de Sousa

Universidade Federal do Ceará · Setembro de 2026



Sobre este documento

Este documento descreve a implementação do simulador: as classes, as estruturas de dados, a estrutura de controle de cada processo e os padrões de projeto utilizados. Os detalhes de cada decisão estão nos ADRs de `docs/adrs/`, e os casos de teste, em `docs/testing.md`.

Sumário

01 Visão geral

02 Organização em pacotes

03 Estrutura de controle do processo (PCB)

04 Estruturas de dados por algoritmo

05 Funcionamento dos algoritmos

06 Desempates e aleatoriedade

07 Métricas e trocas de contexto

08 Entrada, saída e erros

09 Interface gráfica

10 Padrões de projeto

11 Testes

12 Decisões registradas

13 Limitações conhecidas

Visão geral

O simulador lê um conjunto de processos da entrada padrão e um arquivo com `quantum` e `aging`, executa os sete algoritmos sobre a mesma entrada e apresenta, para cada um:

Turnaround médio (`tt`)

Tempo médio entre a criação e o término.

Espera média (`tw`)

Tempo médio pronto, fora da CPU.

Trocas de contexto

Substituições diretas de um processo por outro.

Diagrama de tempo

Uma linha por segundo de simulação.

Há duas formas de uso sobre o mesmo núcleo:

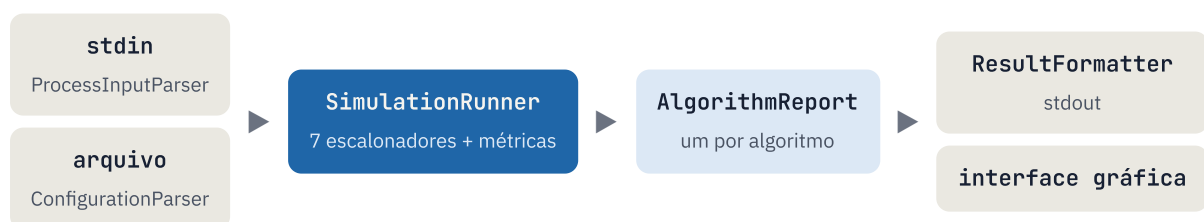
Forma	Ponto de entrada	Execução
Terminal	<code>br.ufc.so.escalonamento.Main</code>	<code>./gradlew -q run --args="config.txt" < processos.txt</code>
Interface gráfica	<code>br.ufc.so.escalonamento.gui.GuiMain</code>	<code>./gradlew runGui</code>

Tecnologia: Java 21, Gradle Wrapper, JUnit 5 e Swing, sem nenhuma dependência de execução externa ao JDK (ADRs 0002 e 0007).

Organização em pacotes

Pacote	Classes	Papel
(raiz)	Main , SchedulerApplication	Contrato de terminal (ADR 0005)
domain	ProcessControlBlock , ProcessState	Modelo do processo
input	ProcessInputParser , ConfigurationParser , SchedulerConfiguration , InputValidationException , ByteOrderMark	Leitura e validação
scheduler	FcfsScheduler , SjfScheduler , SrtfScheduler , NonPreemptivePriorityScheduler , PreemptivePriorityScheduler , RoundRobinScheduler , PriorityRoundRobinScheduler , SchedulingResult , SchedulingSupport	Os sete algoritmos
metrics	MetricsCalculator , SchedulingMetrics	Cálculo das métricas
simulation	SimulationRunner , AlgorithmReport	Orquestração
output	ResultFormatter	Saída textual
gui	GuiMain , SchedulerWindow , AlgorithmPanel , ComparisonPanel , GanttChartPanel , TimelinePresentation , modelos de tabela	Interface gráfica

Fluxo de uma execução pelo terminal



Dependências entre pacotes:

- `scheduler` não conhece entrada, saída nem interface (regra 1 de `architecture.md`);
- `simulation` apenas orquestra;
- `output` e `gui` apenas apresentam resultados.

Estrutura de controle do processo (PCB)

Cada processo é representado por `domain.ProcessControlBlock`, que reúne sua identificação, seus tempos, suas prioridades e seu estado.

Campo	Tipo	Mutável	Significado
<code>id</code>	<code>int</code>	não	Identificador <code>P1</code> , <code>P2</code> ... na ordem das linhas da entrada.
<code>arrivalTime</code>	<code>int</code>	não	Instante de criação.
<code>duration</code>	<code>int</code>	não	Tempo total de CPU necessário, em segundos.
<code>staticPriority</code>	<code>int</code>	não	Prioridade estática; maior número = maior prioridade (notas de aula).
<code>remainingTime</code>	<code>int</code>	sim	Tempo de CPU que ainda falta; começa igual a <code>duration</code> .
<code>dynamicPriority</code>	<code>int</code>	sim	Prioridade dinâmica; começa igual à estática e recebe aging.
<code>state</code>	<code>ProcessState</code>	sim	<code>NEW</code> , <code>READY</code> , <code>RUNNING</code> ou <code>TERMINATED</code> .
<code>completionTime</code>	<code>Integer</code>	sim	Instante de conclusão; <code>null</code> até o término.

Transições permitidas



`RUNNING` volta a `READY` por `markReady()` na preempção ou no fim do quantum. Todas as transições validam o estado de origem.

Operações auxiliares:

- `executeOneSecond()`: decrementa o tempo restante; só é permitida em `RUNNING`.
- `applyAging(taxa)`: incrementa a prioridade dinâmica; só é permitida em `READY`.
- `resetDynamicPriority()`: restaura a prioridade estática; só é permitida em `RUNNING`.
- `freshCopy()`: cria um PCB novo, em `NEW`, com os mesmos atributos de identidade.

Transições validadas

Um erro de algoritmo, como executar um processo que não está em `RUNNING` ou envelhecer o processo em execução, lança exceção e aparece nos testes, em vez de produzir um resultado errado em silêncio.

Sem estado de bloqueio

O enunciado não modela entrada e saída.

Cópias por algoritmo

Cada escalonador chama `freshCopy()` sobre a entrada: os sete recebem os mesmos processos sem compartilhar estado mutável, e a lista original nunca é alterada.

Estruturas de dados por algoritmo

Algoritmo	Classe	Fila de prontos	Processo em execução	Seleção
FCFS	FcfsScheduler	ArrayList	variável local no laço interno	menor criação
SJF	SjfScheduler	ArrayList	variável local no laço interno	menor duração
SRTF	SrtfScheduler	ArrayList	variável <code>runningProcess</code> , fora da lista	menor tempo restante, reavaliado a cada segundo
Prioridade sem preempção	NonPreemptive-PriorityScheduler	ArrayList	variável local no laço interno	maior prioridade estática
Prioridade com preempção	Preemptive-PriorityScheduler	ArrayList	variável <code>runningProcess</code> , fora da lista	maior prioridade estática, reavaliada a cada segundo
Round-Robin	RoundRobinScheduler	ArrayDeque (FIFO)	variável local no laço do quantum	cabeça da fila
Round-Robin com prioridade e aging	PriorityRound-RobinScheduler	ArrayList	variável local no laço do quantum	maior prioridade dinâmica, a cada quantum

Justificativas

ArrayList como fila de prontos

Nos algoritmos com critério de seleção, a escolha percorre todos os prontos em três passagens explícitas: melhor critério, menor tempo restante e empatados. Uma fila de prioridade (`PriorityQueue`) esconderia o critério em um comparador e não resolveria o desempate aleatório nem a preferência por quem já ocupa a CPU. Com as entradas do trabalho, o custo linear é irrelevante (regra 7 de `architecture.md`).

ArrayDeque no Round-Robin

Na versão sem prioridade, a ordem de chegada à fila é a própria regra de seleção:

`pollFirst()` escolhe e `addLast()` recoloca o processo preemptado.

Processo em execução fora da lista

Nos preemptivos, separar `runningProcess` dos prontos torna visíveis a comparação que decide a preempção e o primeiro critério de desempate do enunciado (manter quem já ocupa a CPU).

Linha do tempo como `List<Integer>`

Cada posição `t` representa o intervalo `[t, t+1)`. O valor é o identificador do processo, ou `SchedulingResult.IDLE (0)` quando a CPU está ociosa.

SEÇÃO 05

Funcionamento dos algoritmos

Todos seguem o mesmo esqueleto, escrito por extenso em cada classe para que a regra de cada um fique visível (ADR 0002):

```
enquanto houver processo não terminado:
    admitir processos cuja criação chegou (NEW → READY)
    se não houver pronto: registrar OCIOSO e avançar 1 s
    [regra específica de seleção, execução e preempção]
```

5.1 FCFS, SJF e prioridade sem preempção

```
selecionado ← melhor pronto segundo o critério
READY → RUNNING
enquanto tempo restante > 0:
    executar 1 s; avançar o relógio; admitir chegadas (sem preempção)
RUNNING → TERMINATED
```

Critérios de seleção:

- **FCFS:** menor instante de criação.
- **SJF:** menor duração. Como nada é interrompido, duração e tempo restante coincidem.
- **Prioridade sem preempção:** maior prioridade estática.

5.2 SRTF e prioridade com preempção

```
a cada segundo:
    admitir chegadas
    se existir pronto estritamente melhor que o processo em execução:
        RUNNING → READY (preempção); devolver à lista de prontos
    se a CPU estiver livre: selecionar o melhor pronto; READY → RUNNING
    executar 1 s; se terminou: RUNNING → TERMINATED
```

"Estritamente melhor" significa:

- **SRTF:** tempo restante menor que o do processo em execução.

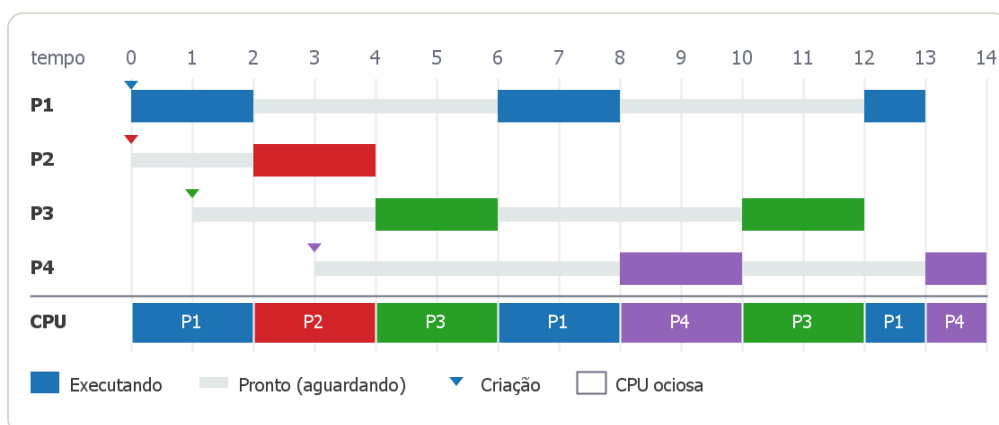
- **Prioridade com preempção:** prioridade estática maior que a do processo em execução.

Em caso de empate, o processo em execução permanece (ADR 0004, regra 12).

5.3 Round-Robin

```
selecionado ← cabeça da fila; READY → RUNNING
repetir até consumir o quantum ou terminar:
    executar 1 s; avançar o relógio; admitir chegadas no final da fila
se terminou: RUNNING → TERMINATED
senão: RUNNING → READY; colocar no final da fila
```

As chegadas que ocorrem exatamente no fim do quantum entram na fila antes do processo preemptado (ADR 0004 e CT-08). A sequência gerada para o caso de referência coincide com o diagrama de exemplo do enunciado.



Round-Robin com quantum 2 no exemplo do enunciado, gerado pela interface: P1, P2, P3, P1, P4, P3, P1, P4.

5.4 Round-Robin com prioridade e aging

```
selecionado ← pronto de maior prioridade dinâmica
READY → RUNNING; prioridade dinâmica ← prioridade estática
repetir até consumir o quantum ou terminar:
    executar 1 s; avançar o relógio; admitir chegadas (sem preempção por prioridade)
se o quantum foi consumido por completo: aging em todos os prontos
admitir quem chegou exatamente neste instante
se terminou: RUNNING → TERMINATED
senão: RUNNING → READY (disputa a próxima seleção sem preferência)
```

Origem das regras:

- **Enunciado:** o aging ocorre a cada quantum, e não há preempção por prioridade.
- **Notas de aula:** o processo selecionado volta à prioridade estática, e os que aguardam são incrementados (diagrama PRIOD).
- **ADR 0004, regras 13 a 15:** os pontos que o enunciado deixa em aberto — preferência de quem esgotou o quantum, término exatamente no limite e chegada exatamente no limite. Cada regra foi validada contra o caso de referência e os CTs 05, 10, 11 e 12.

SEÇÃO 06

Desempates e aleatoriedade

Ordem de desempate aplicada por todos os algoritmos com critério de seleção:

1

Processo que já ocupa a CPU

Implementado pela preempção somente diante de candidato estritamente melhor. Nos não preemptivos, não há processo na CPU no momento da escolha.

2

Menor tempo restante

Segunda passagem sobre os empatados.

3

Escolha aleatória

`SchedulingSupport.chooseRandomly` ordena os candidatos por identificador e sorteia com um `java.util.Random`.

A fonte aleatória é injetada no construtor de cada escalonador:

- na execução normal, `new Random()` ;
- nos testes, `new Random(0)` , o que torna os resultados reproduzíveis (ADR 0004);
- na interface gráfica, semente fixa opcional para repetir demonstrações.

No Round-Robin sem prioridade, a ordem FIFO define a escolha, e o sorteio nunca é usado.

SEÇÃO 07

Métricas e trocas de contexto

Calculadas por `metrics.MetricsCalculator` :

Métrica	Fórmula
Turnaround de um processo	<code>conclusão - criação</code>
Espera de um processo	<code>turnaround - duração</code>
<code>tt</code> e <code>tw</code>	médias aritméticas, sem arredondamento interno
Trocas de contexto	pares de segundos consecutivos executados por processos diferentes

Não contam como troca a carga inicial, a entrada ou a saída de `OCIOSO` e a continuidade do mesmo processo depois de uma decisão (ADR 0004). As médias são apresentadas com duas casas decimais somente na saída.

Entrada, saída e erros

Entrada (ADR 0005)

- o único argumento é o caminho da configuração;
- os processos vêm de `stdin`, três inteiros por linha, separados por espaços ou tabulações;
- a entrada é lida em UTF-8, e um BOM inicial é aceito.

Saída (ADR 0006)

- sete seções, na ordem do enunciado, separadas por uma linha vazia;
- `##` indica o processo em execução, e `--` indica quem não executa naquele segundo.

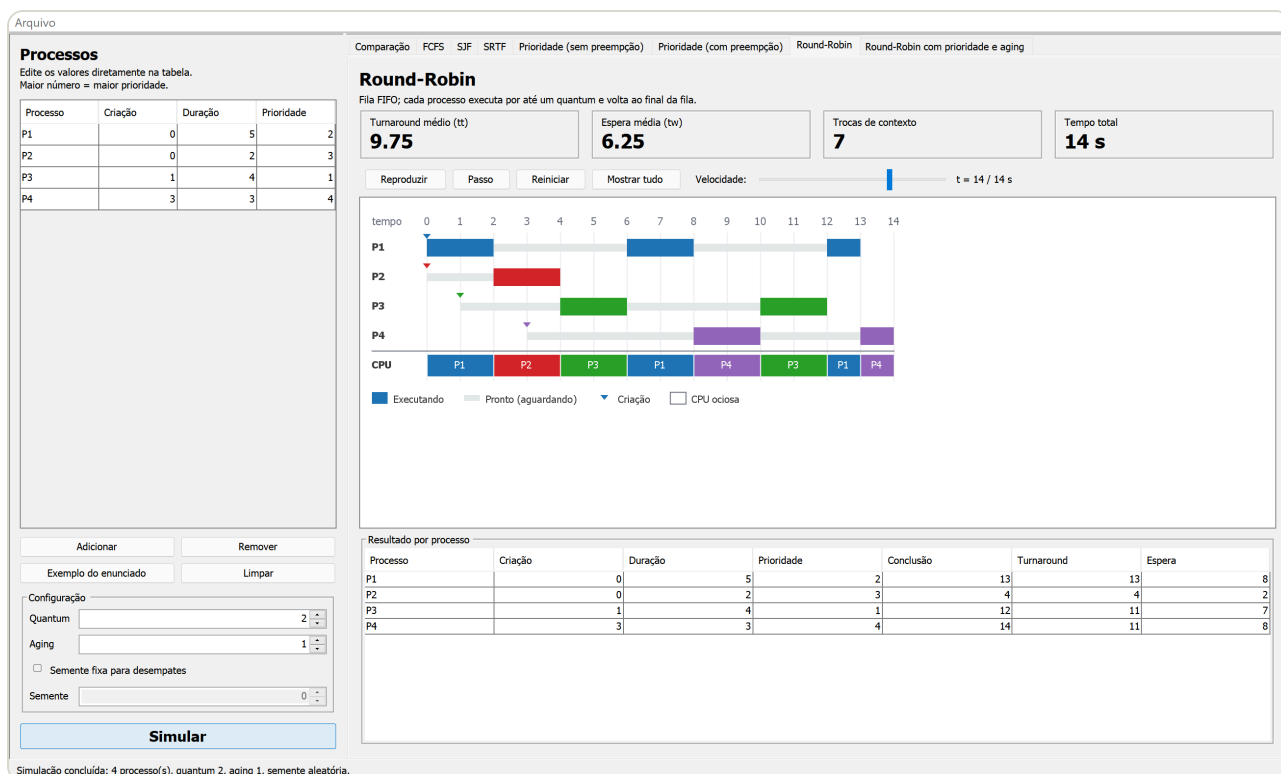
Erros

- mensagem em `stderr`;
- `stdout` vazio;
- código `1` para entrada ou configuração inválida e `2` para uso incorreto.

A saída só é escrita depois de montada por completo, e nunca há resultado parcial.

Interface gráfica

A interface (`gui`, ADR 0007) usa exatamente a mesma lista de `AlgorithmReport` da saída textual. Ela não contém regras de seleção, preempção, aging ou métricas.



Tela principal: entrada editável à esquerda; à direita, a aba do Round-Robin com métricas, diagrama de Gantt e resultado por processo.

- **Entrada:** tabela editável, validada pelo mesmo `ProcessInputParser`; importação de arquivos de processos e de configuração; campos de quantum e aging e opção de semente fixa.
- **Comparação:** tabela com os sete algoritmos, com o melhor valor em negrito, e gráfico de barras de `tt` e `tw`.
- **Aba por algoritmo:** diagrama de Gantt com uma linha por processo e uma da CPU, distinguindo não criado, pronto, executando e terminado; exibição passo a passo; métricas e resultado por processo.

`TimelinePresentation` deriva o que é desenhado a partir do resultado já calculado. A exibição passo a passo apenas revela uma linha do tempo pronta.

SEÇÃO 10

Padrões de projeto

Os padrões foram adotados somente onde resolvem um problema concreto (ADR 0002, item 7, e regra 6 de `architecture.md`).

Padrão ou técnica	Onde	Motivo
Objeto de valor imutável (record)	<code>SchedulerConfiguration</code> , <code>SchedulingResult</code> , <code>SchedulingMetrics</code> , <code>AlgorithmReport</code> , <code>Segment</code>	Dados validados na construção e compartilháveis sem risco de alteração.
Máquina de estados explícita	<code>ProcessControlBlock</code> + <code>ProcessState</code>	Torna as transições do processo verificáveis e fáceis de explicar.
Injeção de dependências por construtor	<code>Random</code> nos escalonadores, <code>Supplier<Random></code> no <code>SimulationRunner</code> , fluxos na <code>SchedulerApplication</code>	Testes determinísticos e testáveis sem processo externo.
Model-View (Swing)	<code>ProcessTableModel</code> , <code>ComparisonTableModel</code> , <code>ProcessResultTableModel</code>	Separa os dados exibidos dos componentes; os modelos são testados sem janela.
Observer (listeners do Swing)	botões, abas, <code>Timer</code> da exibição	Mecanismo padrão do Swing para eventos.

Padrões deliberadamente não usados

Strategy / interface comum de escalonadores

Uma abstração `Scheduler` com um laço genérico e políticas plugáveis reduziria linhas, mas esconderia as diferenças teóricas: seleção uma vez versus a cada segundo, preempção versus quantum, fila FIFO versus seleção por critério. O ADR 0002 veda essa generalização. O `SimulationRunner` chama os sete algoritmos explicitamente.

Template Method

Pelo mesmo motivo. O que é realmente comum (cópia, admissão e sorteio) está em `SchedulingSupport`, como funções simples.

SEÇÃO 11

Testes

A suíte JUnit 5 (`./gradlew clean test`) tem mais de 800 testes:

Tipo	O que verifica
Unidade	Parsers, PCB, cada algoritmo (caso de referência e CTs específicos), métricas, formatação e modelos da interface.
Integração	Saída completa do caso de referência comparada com <code>src/test/resources/cases/reference/expected-output.txt</code> , arquivo gerado a partir da tabela de oráculos e não pelo simulador; execução de todos os casos de <code>cases/</code> ; contrato de erro.
Propriedades	Centenas de entradas geradas com semente fixa, verificando as invariantes do CT-18, as métricas contra um cálculo independente, a otimalidade do SRTF para o turnaround médio e o determinismo.
Interface	Renderização do diagrama em memória (modo headless), com verificação da cor das células.

Os resultados esperados foram definidos antes das implementações (`docs/testing.md`). O relatório da validação integrada está em `docs/testing.md` §9.

SEÇÃO 12

Decisões registradas

ADR	Assunto
0001	Governança e precedência das fontes
0002	Java 21, Gradle, JUnit 5 e visibilidade dos algoritmos
0003	Adoção da interface gráfica
0004	Convenções de simulação, desempates e aging
0005	Contrato de entrada, validação e erros
0006	Formato da saída textual
0007	Swing como tecnologia da interface
0008	Conteúdo da entrega

Os ADRs completos estão em `docs/adrs/`.

SEÇÃO 13

Limitações conhecidas

- O tempo é discreto, em segundos inteiros, como na entrada do enunciado.
- O custo da troca de contexto não é simulado; as trocas são apenas contadas.
- Não há operações de entrada e saída nem estado bloqueado.

- A escolha aleatória, fora dos testes, pode variar entre execuções.
- A janela e os botões da interface são verificados manualmente; a lógica de apresentação e o desenho do diagrama têm testes automatizados.